

C++ Midterm Practice: 100 Questions

Savitch, Problem Solving with C++, Chapters 1-5 - Multiple choice - Highlighted study version with inline explanations

Name: _____

Date: _____

Score: ____ / 100

Correct answer: the correct multiple-choice option is highlighted in light green. The short Why note follows each question and is based on the provided answer key.

CHAPTER 1

Computers and C++ Programming

Questions 1-15

1. Which of these is NOT one of the five main components of a computer?

A. Processor (CPU)

B. Main memory

C. Input devices

D. Compiler

Why: The five components are input devices, output devices, processor, main memory, and secondary memory. A compiler is software.

2. A binary digit (bit) is:

A. The address of a memory location

B. A digit that can only be 0 or 1

C. A group of eight binary values

D. A single character such as 'A'

Why: A bit holds only 0 or 1. Eight bits make a byte.

3. How many bits are in one byte?

A. 4

B. 32

C. 8

D. 16

Why: Each memory location (byte) holds eight bits.

4. A data item that is too large for one byte is stored in several consecutive bytes. Its address refers to:

A. Every byte equally

B. The middle byte

C. The last byte

D. The first byte

Why: The address of a multi-byte item is the address of its first byte.

5. The same bit pattern 01000001 could mean 'A', 65, or an instruction. How does the computer decide?

- | | |
|---|---|
| A. By the file extension | B. The operating system labels every byte |
| C. By the current instruction that uses it | D. The pattern always means 'A' |

Why: Interpretation depends on the instruction currently operating on that memory.

6. Which memory holds instructions and data between sessions (after power is off)?

- | | |
|----------------------------|------------------|
| A. Main memory | B. CPU registers |
| C. Secondary memory | D. Cache |

Why: Secondary memory (disk, etc.) is the permanent record; main memory holds the running program.

7. "Random access" memory means:

- | | |
|--|---------------------------------------|
| A. Data is stored in random order | B. Data must be searched sequentially |
| C. The computer can directly access any memory location | D. Contents change randomly |

Why: RAM allows direct access to any location. Sequential access requires passing other items first.

8. A compiler translates:

- | | |
|---|--|
| A. A high-level language program into machine language | B. Machine language into a high-level language |
| C. Assembly into a high-level language | D. Object code into source code |

Why: The compiler turns source code (high-level) into object code (machine language).

9. The original program written in a high-level language is called the:

- | | |
|-----------------|-----------------------|
| A. Executable | B. Source code |
| C. Machine code | D. Object code |

Why: Source code is the original; object code is the translated version.

10. What does a linker do?

- | | |
|---|---------------------------------|
| A. Finds syntax errors | B. Loads the operating system |
| C. Combines your object code with object code for precompiled routines (such as I/O) into a runnable program | D. Translates C++ into assembly |

Why: The linker joins your object code with already-compiled library code.

11. Who is considered the first programmer?

- | | |
|---|----------------------|
| A. Grace Hopper | B. Charles Babbage |
| C. Ada Augusta, Countess of Lovelace | D. Bjarne Stroustrup |

Why: Ada Lovelace worked with Babbage, who designed the first programmable computer.

12. An algorithm is:

- | | |
|--|---|
| A. Any C++ source file | B. A sequence of precise instructions that leads to a solution |
| C. A program written in machine language | D. A type of compiler |

Why: A program is an algorithm expressed in a language the computer understands.

13. The two main phases of program design are:

- | | |
|----------------------------|--|
| A. Input and output | B. Problem-solving phase and implementation phase |
| C. Testing and maintenance | D. Compiling and linking |

Why: Problem solving produces the algorithm; implementation translates it into a programming language.

14. In OOP, a single name having multiple meanings depending on its context is called:

- | | |
|----------------|------------------------|
| A. Inheritance | B. Encapsulation |
| C. Abstraction | D. Polymorphism |

Why: Encapsulation is information hiding; inheritance is reuse; polymorphism is one name, many meanings.

15. A program compiles and runs without crashing but prints the wrong answer. This is a:

- | | |
|-------------------|-----------------------|
| A. Linker error | B. Logic error |
| C. Run-time error | D. Syntax error |

Why: Logic errors are the hardest: the compiler and computer can't detect them.

C++ Basics

16. Which is a valid C++ identifier?

- | | |
|-------------|-------------------|
| A. double | B. my-total |
| C. 2ndValue | D. _total2 |

Why: Must start with a letter or underscore, contain only letters, digits, underscores, and not be a keyword.

17. Keywords (reserved words) in C++:

- | | |
|--|---|
| A. Can be used as variable names if capitalized the same way | B. Cannot be used as identifiers |
| C. Must be declared before use | D. Are only used in comments |

Why: Keywords like int, double, if have fixed meanings and cannot be identifiers.

18. What does `numberOfBars = numberOfBars + 3; do?`

- | | |
|--|---------------------------|
| A. It is a compile error because it can't be true | B. Compares the two sides |
| C. Sets numberOfBars to its previous value plus 3 | D. Sets numberOfBars to 3 |

Why: = is assignment, not equality.

19. After `double mpg(26.3), area(0.0), volume;` which statement is true?

- | | |
|---|------------------------------|
| A. mpg is 26.3, area is 0.0, volume is uninitialized | B. volume is 0.0 by default |
| C. It is a syntax error | D. All three are initialized |

Why: Parentheses form initializes; volume is only declared and holds a garbage value.

20. What is printed?

```
cout << 17 / 5 << " " << 17 % 5;
```

- | | |
|---------------|----------|
| A. 3 2 | B. 3.4 0 |
| C. 3 3 | D. 3.4 2 |

Why: Integer division discards the fraction (3); % gives the remainder (2).

21. What value does d hold?

```
double d = 11 / 2;
```

A. 5.0

B. Compile error

C. 6.0

D. 5.5

Why: $11 / 2$ is integer division = 5, then stored as 5.0.

22. What is the value of $7.0 / 2$?

A. 3.5

B. 4

C. 3

D. 3.0

Why: If either operand is double, floating-point division is used.

23. What is the value of $10 / 4 * 4$?

A. 1

B. 8

C. 10

D. 2.5

Why: / and * have equal precedence, left to right: $(10/4)*4 = 2*4 = 8$.

24. What does i hold?

```
int i = 7.9;
```

A. Compile error

B. 7

C. 7.9

D. 8

Why: Assigning a double to an int truncates the fractional part (no rounding).

25. What is the value of n ?

```
int n = 4;  
n *= 2 + 3;
```

A. 14

B. 20

C. 11

D. 9

Why: $n *= \text{expr}$ means $n = n * (\text{expr}) = 4 * 5 = 20$.

26. Which escape sequence produces a tab?

A. \"

B. \n

C. \

D. \t

Why: \n newline, \t tab, \ backslash, \" double quote.

27. Which three statements make cout always show exactly two decimal places for doubles?

A. cout.precision(2); only

B. cout << fixed2;

**C. cout.setf(ios::fixed);
cout.setf(ios::showpoint); cout.precision(2);**

D. cout.setf(ios::decimal, 2);

Why: fixed selects fixed-point notation, showpoint forces the decimal point, precision(2) sets two digits.

28. #include <iostream> is needed to:

A. Use the string class

B. Enable math functions

C. Declare main

D. Make the definitions of cin and cout available

Why: iostream holds cin/cout; using namespace std makes the names available.

29. With cin >> v1 >> v2 >> v3; the user's three values should be separated by:

A. Semicolons

B. Tabs only

C. Spaces (whitespace)

D. Commas

Why: Values are whitespace-separated and not read until Enter is pressed.

30. Which is a constant of type char ?

A. 'A'

B. A

C. "A"

D. char(A)

Why: char constants use single quotes; double quotes make a string.

31. To use the string class you must include:

A. <string>

B. <cstring> only

C. <iostream>

D. <cmath>

Why: #include along with using namespace std.

32. 5.89e-6 equals:

- | | |
|--------------|----------------------|
| A. 5890000 | B. 0.00000589 |
| C. 0.0000589 | D. -5.89 |

Why: e-6 moves the decimal point six places left.

33. What is printed?

```
int x = 1;
while (x < 20)
x = x * 2;
cout << x;
```

- | | |
|------------------|--------------|
| A. 16 | B. 32 |
| C. Infinite loop | D. 20 |

Why: x: 1, 2, 4, 8, 16, 32. The loop stops when x = 32.

34. What is printed?

```
int n = 10;
do {
cout << n << " ";
n++;
} while (n < 5);
```

- | | |
|------------------|-------------------|
| A. Nothing | B. 10 |
| C. Infinite loop | D. 10 11 12 13 14 |

Why: A do-while body always runs at least once; then $n < 5$ is false.

35. What is wrong with `if (x = 5) cout << "five";` ?

- | | |
|-------------------------|--|
| A. It is a syntax error | B. It assigns 5 to x, so the condition is always true |
| C. Nothing | D. It compares x to 5 twice |

Why: `=` assigns; `==` compares. A nonzero value counts as true.

36. Given `const double PI = 3.14159;` , what happens with `PI = 3.14;` later?

- | | |
|-------------------------|-----------------------------------|
| A. Run-time error | B. PI becomes 3.14 |
| C. Compile error | D. Warning only; PI stays 3.14159 |

Why: A const named constant cannot be changed after initialization.

37. Which type holds only the values true and false ?

A. enum

B. int

C. bool

D. char

Why: bool is the Boolean type.

More Flow of Control

Questions 38-59

38. If y is 8, what is $!(y < 3) || (y > 7)$?

- | | |
|------------------|-----------------|
| A. true | B. false |
| C. Compile error | D. 8 |

Why: $!(\text{false} || \text{true}) = !(\text{true}) = \text{false}$.

39. Without parentheses, the default precedence order (highest first) is:

- | | |
|--|-------------------------------------|
| A. $ $, $\&\&$, relational, $!$ | B. relational, $!$, $ $, $\&\&$ |
| C. $!$, relational, $\&\&$, $$ | D. $\&\&$, $ $, $!$, relational |

Why: $!$ first, then $< >$ etc., then $\&\&$, then $||$.

40. With $kids = 0$, what happens in $\text{if}((kids != 0) \&\& (pieces / kids \geq 2))$?

- | | |
|---|------------------------------------|
| A. Short-circuit evaluation skips the division; condition is false | B. Division-by-zero run-time error |
| C. Condition is true | D. Compile error |

Why: The left side is false, so $\&\&$ doesn't evaluate the right side.

41. With $time = 36$ and $limit = 60$, what is $!(time > limit)$?

- | | |
|--|--|
| A. true, because 36 is not greater than 60 | B. false, because it is evaluated as $!(time) > limit$, i.e. $0 > 60$ |
| C. Undefined | D. A compile error |

Why: $!$ has higher precedence than $>$. Write $!(time > limit)$ if that's what you mean.

42. When an integer is used as a Boolean value:

- | | |
|---|-----------------------|
| A. Any nonzero value is true; 0 is false | B. It causes an error |
| C. Negative numbers are false | D. Only 1 is true |

Why: C++ treats nonzero as true and zero as false.

43. Given enum Direction {NORTH, SOUTH, EAST, WEST}; what is the value of EAST?

A. 'E'

B. 1

C. 2

D. 3

Why: Values start at 0 by default: NORTH 0, SOUTH 1, EAST 2, WEST 3.

44. Given enum MyEnum {ONE = 17, TWO, THREE, FOUR = -3, FIVE}; what is FIVE?

A. 20

B. -2

C. 4

D. -4

Why: Each unspecified constant is 1 more than the previous: FOUR is -3, so FIVE is -2.

45. What is printed when fuel is 0.5?

```
if (fuel < 0.75)
if (fuel < 0.25)
cout << "Fuel very low.";
else
cout << "Fuel over 3/4.";
```

A. Both messages

B. Nothing

C. Fuel over 3/4.

D. Fuel very low.

Why: The else pairs with the nearest previous if (fuel < 0.25). Use braces to fix it.

46. What is printed?

```
int n = 2;
switch (n) {
case 1: cout << "A";
case 2: cout << "B";
case 3: cout << "C"; break;
default: cout << "D";
}
```

A. BC

B. B

C. BCD

D. ABC

Why: Without break, execution falls through from case 2 into case 3, then break stops it.

47. The controlling expression of a switch may NOT be of type:

A. int

B. char

C. double

D. bool

Why: It must be an integral type such as int, char, bool, or an enum. Not double or string.

48. In a switch, the default case runs when:

- | | |
|---------------------------------|---------------------------|
| A. It always runs last | B. A break is missing |
| C. No case label matches | D. The first case matches |

Why: default handles all values not listed in a case.

49. What is printed?

```
for (int i = 0; i < 10; i += 3)
    cout << i << " ";
```

- | | |
|----------|-------------------|
| A. 0 3 6 | B. 0 3 6 9 |
| C. 3 6 9 | D. 0 3 6 9 12 |

Why: i takes 0, 3, 6, 9; then 12 fails $i < 10$.

50. How many times does the body of `for (int i = 1; i <= 5; i++)` execute?

- | | |
|-------------|-------------|
| A. 5 | B. 6 |
| C. 4 | D. Infinite |

Why: $i = 1, 2, 3, 4, 5$.

51. After this code, what are m and n?

```
int n = 5;
int m = n++ + 2;
```

- | | |
|-------------------|-------------------------------------|
| A. $m = 8, n = 5$ | B. $m = 7, n = 6$ |
| C. $m = 8, n = 6$ | D. $m = 7, n = 5$ |

Why: `n++` returns the old value (5) then increments. $m = 5 + 2 = 7$, $n = 6$.

52. What is printed?

```
for (int i = 0; i < 5; i++);
cout << "hi";
```

- | | |
|-----------------------|---------------------------|
| A. hi printed 5 times | B. hi printed once |
| C. Nothing | D. Compile error |

Why: The stray semicolon is an empty loop body. `cout` runs once after the loop.

53. What is printed?

```
int t = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;
    if (i > 7) break;
    t += i;
}
cout << t;
```

A. 25

B. 55

C. 9

D. 16

Why: Even i are skipped. Odd i: $1+3+5+7 = 16$, then $i = 9$ triggers break.

54. The break statement inside a loop:

A. Ends the loop immediately

B. Skips to the next iteration

C. Ends the program

D. Restarts the loop

Why: break exits the loop; continue skips the rest of the current iteration.

55. Which loop is the natural choice when the body must execute at least once?

A. while

B. for

C. switch

D. do-while

Why: do-while tests its condition after the body.

56. A loop that stops when it reads a special value that cannot be valid data uses a:

A. Counter

B. Sentinel value

C. Flag

D. Break-point

Why: E.g. reading scores until a negative number is entered.

57. How many stars are printed?

```
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        cout << "*";
```

A. 12

B. 9

C. 16

D. 7

Why: The inner loop runs 4 times for each of 3 outer iterations: $3 \times 4 = 12$.

58. What is printed?

```
int sum = 0;
for (int i = 1; i <= 4; i++)
    sum += i;
cout << sum;
```

A. 15

B. 6

C. 10

D. 4

Why: $1 + 2 + 3 + 4 = 10$.

59. What happens?

```
int x = 10;
while (x != 0)
    x -= 3;
```

A. Compile error

B. Loop runs 3 times

C. Infinite loop

D. Loop runs 4 times

Why: x goes 10, 7, 4, 1, -2, ... and never equals 0. Prefer $x > 0$ in such tests.

Procedural Abstraction & Functions That Return a Value

Questions 60-80

60. Top-down design is also called:

- | | |
|-------------------------------|---------------------------|
| A. Bottom-up design | B. Overloading |
| C. Stepwise refinement | D. Procedural abstraction |

Why: Break the task into subtasks, then refine each until trivial.

61. In the `Root = sqrt(9.0);` the value 9.0 is the function's:

- | | |
|-------------------|--------------------|
| A. Return value | B. Argument |
| C. Parameter type | D. Prototype |

Why: `sqrt(9.0)` is a function call with argument 9.0; it returns 3.0.

62. Which library must be included to use `sqrt` and `pow` ?

- | | |
|--------------------------------------|----------------------------------|
| A. <code><cstdlib></code> | B. <code><iostream></code> |
| C. <code><cmath></code> | D. <code><ctime></code> |

Why: `cmath` holds `sqrt`, `pow`, `fabs`, `ceil`, `floor`.

63. Which function returns the absolute value of a double ?

- | | |
|-----------------------------|----------------------|
| A. <code>dabs</code> | B. <code>abs</code> |
| C. <code>fabs</code> | D. <code>labs</code> |

Why: `abs` works on `int` (`cstdlib`); `fabs` on `double` (`cmath`).

64. What is `floor(5.8) + ceil(5.2)` ?

- | | |
|-------|--------------|
| A. 10 | B. 11 |
| C. 12 | D. 5.8 |

Why: `floor(5.8) = 5`, `ceil(5.2) = 6`, total 11.

65. What is `7 / abs(-2)` ?

A. 3.5

B. -3

C. 3

D. 4

Why: `abs(-2)` is int 2, and `7 / 2` is integer division = 3.

66. What is `pow(2, 3)` ?

A. 9

B. 8

C. 6

D. 5

Why: 2 raised to the 3rd power is 8.

67. Which expression simulates rolling a six-sided die?

A. `rand(6)`

B. `rand() % 6`

C. `(rand() % 6) + 1`

D. `rand() % 7`

Why: `rand() % 6` gives 0-5; adding 1 gives 1-6.

68. `srand(time(0));` should be called:

A. Before every call to `rand()`

B. After every loop

C. Never; `rand()` seeds itself

D. Only once, to seed the random number generator

Why: Seed once so each run produces a different pseudo-random sequence.

69. With `int totalCandy = 9, numberOfPeople = 4;` which gives 2.25?

A. `static_cast<double>(totalCandy / numberOfPeople)`

B. `static_cast<double>(totalCandy) / numberOfPeople`

C. `double(totalCandy / numberOfPeople)`

D. `totalCandy / numberOfPeople`

Why: Cast before dividing. Casting the result of `9/4` gives 2.0 because integer division already happened.

70. A function declaration (prototype):

A. Must appear after main

B. Contains the function body

C. Ends with a semicolon and must appear before the function is called

D. Is only needed for void functions

Why: The prototype tells the compiler the return type, name, and parameter types.

71. Which statement is true?

A. double totalCost(int, double); is a valid declaration

B. Function headers may omit parameter names

C. A function may be declared only once per program and never defined

D. Declarations must list parameter names

Why: Declarations may list only types; the definition's header must name the parameters.

72. What is printed?

```
int f(int n) {  
    n = n * 2;  
    return n + 1;  
}  
// in main:  
int x = 3;  
int y = f(x);  
cout << x << " " << y;
```

A. 6 7

B. 3 7

C. 6 13

D. 3 6

Why: Call-by-value: n is a copy, so x stays 3. $y = 3 * 2 + 1 = 7$.

73. With call-by-value parameters:

A. The argument variable changes when the parameter changes

B. The & symbol is required

C. Arguments must be variables

D. The formal parameter receives a copy of the argument's value

Why: Changes to the formal parameter don't affect the argument.

74. Given `char grade(int received, int minScore);` the call `grade(minScore, received)` :

A. Swaps them automatically

B. Is caught by the compiler

C. Causes a run-time crash

D. Compiles but gives a wrong result because arguments are in the wrong logical order

Why: The compiler checks types and count, not logical order.

75. The return statement in a value-returning function:

A. Only works in main

B. Prints the value

C. Ends the function call and sends back a value

D. Is optional

Why: Each branch may have its own return.

76. A function that returns bool can be used:

A. Only inside loops

B. Only with call-by-reference

C. Only in cout statements

D. Wherever a Boolean expression is expected, such as an if condition

Why: e.g. if (appropriate(rate)) ... reads more clearly than the full expression.

77. Procedural abstraction means a function should be used like a:

A. Global variable

B. Black box: you need to know what it does, not how

C. Loop

D. Header file

Why: The declaration and its comment should be all a user needs.

78. A variable declared inside a function body is:

A. A named constant

B. Local to that function

C. Shared with main

D. Global

Why: Local variables exist only within the function; same names elsewhere are different variables.

79. A named constant declared outside all functions (e.g. const double PI = 3.14159; above main) is:

A. Illegal

B. Local to main

C. A global named constant, usable by all functions after it

D. Visible only to the next function

Why: Global named constants are common and safe; global variables are discouraged.

80. Two overloaded functions must differ in:

A. Parameter names only

B. The comments above them

C. Number and/or types of parameters

D. Return type only

Why: Functions that differ only in return type cannot be overloaded.

Functions for All Subtasks

Questions 81-100

81. A void function:

- | | |
|------------------------------|----------------------------|
| A. Cannot have parameters | B. Cannot contain return |
| C. Returns an int by default | D. Returns no value |

Why: void replaces the return type; return; with no expression may end it early.

82. Given void showResults(double f, double c); which call is correct?

- | | |
|------------------------------------|---------------------------------------|
| A. cout << showResults(32.5, 0.3); | B. double r = showResults(32.5, 0.3); |
| C. showResults(32.5, 0.3); | D. return showResults(32.5, 0.3); |

Why: A void function call is a statement by itself, not part of an expression.

83. Why might a void function contain return; ?

- | | |
|--------------------------------|---|
| A. To return the last variable | B. It is required at the end |
| C. To return 0 | D. To end the function early, e.g. to avoid division by zero |

Why: If omitted, an implicit return happens at the end of the body.

84. What is the bug in celsius = (5 / 9) * (fahrenheit - 32); ?

- | | |
|---------------------------|---|
| A. fahrenheit must be int | B. Missing parentheses |
| C. No bug | D. 5 / 9 is integer division and equals 0, so the result is always 0 |

Why: Write 5.0 / 9 to get floating-point division.

85. The & in void getInput(double& f); means:

- | | |
|--|--------------------|
| A. f is a call-by-reference parameter | B. f is global |
| C. f is a pointer | D. f is a constant |

Why: The & goes in both the declaration and the definition.

86. The argument for a call-by-reference parameter must be:

- | | |
|----------------------|------------------|
| A. A variable | B. A number |
| C. A constant | D. An expression |

Why: The function receives the memory location of the argument, so it must be a variable.

87. What is printed?

```
void swapValues(int& a, int& b) {  
    int t = a; a = b; b = t;  
}  
// in main:  
int x = 4, y = 9;  
swapValues(x, y);  
cout << x << " " << y;
```

- | | |
|---------------|--------|
| A. 9 4 | B. 4 4 |
| C. 9 9 | D. 4 9 |

Why: Reference parameters change the arguments themselves.

88. The & was left off both parameters. What is printed?

```
void swapValues(int a, int b) {  
    int t = a; a = b; b = t;  
}  
// in main:  
int x = 4, y = 9;  
swapValues(x, y);  
cout << x << " " << y;
```

- | | |
|---------------|------------------|
| A. 9 4 | B. Compile error |
| C. 4 9 | D. 9 9 |

Why: Forgetting & makes them call-by-value: inadvertent local variables. x and y don't change.

89. In void goodStuff(int& p1, int p2, double& p3); which parameters can change the caller's variables?

- | | |
|------------|---------------------|
| A. p1 only | B. p1 and p3 |
| C. p2 only | D. All three |

Why: Only reference parameters (with &) affect the arguments.

90. What is printed?

```
void f(int& a, int b) {  
    a = a + b;  
    b = b * 2;  
}  
// in main:  
int x = 2, y = 5;  
f(x, y);  
cout << x << " " << y;
```

A. 7 10

B. 7 5

C. 2 10

D. 2 5

Why: a is a reference to x (becomes 7); b is a copy of y, so y stays 5.

91. What is printed?

```
void g(int& a, int& b) {  
    a = a + 1;  
    b = b + 1;  
}  
// in main:  
int n = 1;  
g(n, n);  
cout << n;
```

A. 1

B. 2

C. 3

D. 4

Why: Both a and b refer to the same variable n, so it is incremented twice.

92. How do you decide between call-by-value and call-by-reference?

A. Use reference if the function must change the argument variable; otherwise value

B. Always use reference for speed

C. Use value for doubles, reference for ints

D. Use reference only in void functions

Why: That is the rule in Display 5.6.

93. A precondition:

A. Is checked by the compiler

B. Is the function's return type

C. Describes the effect after the call

D. States what is assumed true when the function is called

Why: The function should not be called unless the precondition holds.

94. A postcondition:

- | | |
|--|---|
| A. Must be written after main | B. Is optional C++ syntax |
| C. Describes what will be true after the call, including the return value and changes to reference parameters | D. Lists the function's local variables |

Why: Write pre/postconditions before designing how the function works.

95. Which is true about functions calling functions?

- | | |
|--|--|
| A. A function body can call another function, whose declaration must appear before the call | B. Functions can't call void functions |
| C. Only main can call functions | D. A function can be defined inside another function |

Why: Functions cannot be nested, but they can call each other.

96. A driver program is:

- | | |
|--|--|
| A. The operating system | B. A program written only to test a single function |
| C. A simplified function that returns fixed values | D. The main program of the final product |

Why: Drivers call the function with test data and display the results.

97. A stub is:

- | | |
|----------------------------------|---|
| A. A function with no parameters | B. A test program for one function |
| C. A broken function | D. A simplified version of a not-yet-tested function, used while testing other functions |

Why: Stubs usually just return test values and are simple enough to trust.

98. The fundamental rule for testing functions says every function should be tested in a program in which:

- | | |
|---|----------------------------|
| A. There are no loops | B. main is empty |
| C. Every other function has already been fully tested and debugged | D. All functions are stubs |

Why: This way any bug found belongs to the function under test.

99. What happens when the boolean in `assert(denominator != 0);` is false?

- | | |
|--------------------------------------|------------------------------|
| A. It prints a warning and continues | B. denominator is set to 1 |
| C. The compiler rejects the program | D. The program aborts |

Why: When the assertion condition is false, `assert` stops the program by aborting it. That is why option D is correct.

100. Which is NOT a recommended debugging technique?

- | | |
|---|--|
| A. Randomly changing code until it works | B. Using <code>cout</code> statements to trace variable values |
| C. Keeping an open mind about where the bug is | D. Using a debugger to step line by line |

Why: Don't change code without understanding what you're doing.

Answer Key

Original correct letters and short explanations retained as a compact reference.

CHAPTER 1

Computers and C++ Programming

Questions 1-15

1. **D** The five components are input devices, output devices, processor, main memory, and secondary memory. A compiler is software.
2. **B** A bit holds only 0 or 1. Eight bits make a byte.
3. **C** Each memory location (byte) holds eight bits.
4. **D** The address of a multi-byte item is the address of its first byte.
5. **C** Interpretation depends on the instruction currently operating on that memory.
6. **C** Secondary memory (disk, etc.) is the permanent record; main memory holds the running program.
7. **C** RAM allows direct access to any location. Sequential access requires passing other items first.
8. **A** The compiler turns source code (high-level) into object code (machine language).
9. **B** Source code is the original; object code is the translated version.
10. **C** The linker joins your object code with already-compiled library code.
11. **C** Ada Lovelace worked with Babbage, who designed the first programmable computer.
12. **B** A program is an algorithm expressed in a language the computer understands.
13. **B** Problem solving produces the algorithm; implementation translates it into a programming language.
14. **D** Encapsulation is information hiding; inheritance is reuse; polymorphism is one name, many meanings.
15. **B** Logic errors are the hardest: the compiler and computer can't detect them.

CHAPTER 2

C++ Basics

Questions 16-37

16. **D** Must start with a letter or underscore, contain only letters, digits, underscores, and not be a keyword.
17. **B** Keywords like int, double, if have fixed meanings and cannot be identifiers.
18. **C** = is assignment, not equality.
19. **A** Parentheses form initializes; volume is only declared and holds a garbage value.
20. **A** Integer division discards the fraction (3); % gives the remainder (2).
21. **A** 11 / 2 is integer division = 5, then stored as 5.0.
22. **A** If either operand is double, floating-point division is used.
23. **B** / and * have equal precedence, left to right: (10/4)*4 = 2*4 = 8.
24. **B** Assigning a double to an int truncates the fractional part (no rounding).
25. **B** n *= expr means n = n * (expr) = 4 * 5 = 20.
26. **D** \n newline, \t tab, \\ backslash, \" double quote.
27. **C** fixed selects fixed-point notation, showpoint forces the decimal point, precision(2) sets two digits.
28. **D** iostream holds cin/cout; using namespace std makes the names available.
29. **C** Values are whitespace-separated and not read until Enter is pressed.
30. **A** char constants use single quotes; double quotes make a string.
31. **A** #include along with using namespace std.
32. **B** e-6 moves the decimal point six places left.
33. **B** x: 1, 2, 4, 8, 16, 32. The loop stops when x = 32.
34. **B** A do-while body always runs at least once; then n < 5 is false.
35. **B** = assigns; == compares. A nonzero value counts as true.

36. **C** A const named constant cannot be changed after initialization.
37. **C** bool is the Boolean type.

CHAPTER 3

More Flow of Control

Questions 38-59

38. **B** `!(false || true) = !(true) = false.`
39. **C** `! first, then < > etc., then &&, then ||.`
40. **A** The left side is false, so `&&` doesn't evaluate the right side.
41. **B** `!` has higher precedence than `>`. Write `!(time > limit)` if that's what you mean.
42. **A** C++ treats nonzero as true and zero as false.
43. **C** Values start at 0 by default: NORTH 0, SOUTH 1, EAST 2, WEST 3.
44. **B** Each unspecified constant is 1 more than the previous: FOUR is -3, so FIVE is -2.
45. **C** The else pairs with the nearest previous if (`fuel < 0.25`). Use braces to fix it.
46. **A** Without break, execution falls through from case 2 into case 3, then break stops it.
47. **C** It must be an integral type such as int, char, bool, or an enum. Not double or string.
48. **C** default handles all values not listed in a case.
49. **B** `i` takes 0, 3, 6, 9; then 12 fails `i < 10`.
50. **A** `i = 1, 2, 3, 4, 5.`
51. **B** `n++` returns the old value (5) then increments. `m = 5 + 2 = 7`, `n = 6`.
52. **B** The stray semicolon is an empty loop body. `cout` runs once after the loop.
53. **D** Even `i` are skipped. Odd `i`: `1+3+5+7 = 16`, then `i = 9` triggers break.
54. **A** break exits the loop; continue skips the rest of the current iteration.
55. **D** do-while tests its condition after the body.
56. **B** E.g. reading scores until a negative number is entered.
57. **A** The inner loop runs 4 times for each of 3 outer iterations: `3 × 4 = 12`.
58. **C** `1 + 2 + 3 + 4 = 10`.
59. **C** `x` goes 10, 7, 4, 1, -2, ... and never equals 0. Prefer `x > 0` in such tests.

CHAPTER 4

Procedural Abstraction & Functions That Return a Value

Questions 60-80

60. **C** Break the task into subtasks, then refine each until trivial.
61. **B** `sqrt(9.0)` is a function call with argument 9.0; it returns 3.0.
62. **C** `cmath` holds `sqrt`, `pow`, `fabs`, `ceil`, `floor`.
63. **C** `abs` works on int (`cstdlib`); `fabs` on double (`cmath`).
64. **B** `floor(5.8) = 5`, `ceil(5.2) = 6`, total 11.
65. **C** `abs(-2)` is int 2, and `7 / 2` is integer division = 3.
66. **B** 2 raised to the 3rd power is 8.
67. **C** `rand() % 6` gives 0-5; adding 1 gives 1-6.
68. **D** Seed once so each run produces a different pseudo-random sequence.
69. **B** Cast before dividing. Casting the result of `9/4` gives 2.0 because integer division already happened.
70. **C** The prototype tells the compiler the return type, name, and parameter types.
71. **A** Declarations may list only types; the definition's header must name the parameters.
72. **B** Call-by-value: `n` is a copy, so `x` stays 3. `y = 3*2 + 1 = 7`.

73.	D	Changes to the formal parameter don't affect the argument.
74.	D	The compiler checks types and count, not logical order.
75.	C	Each branch may have its own return.
76.	D	e.g. if (appropriate(rate)) ... reads more clearly than the full expression.
77.	B	The declaration and its comment should be all a user needs.
78.	B	Local variables exist only within the function; same names elsewhere are different variables.
79.	C	Global named constants are common and safe; global variables are discouraged.
80.	C	Functions that differ only in return type cannot be overloaded.

CHAPTER 5

Functions for All Subtasks

Questions 81-100

81.	D	void replaces the return type; return; with no expression may end it early.
82.	C	A void function call is a statement by itself, not part of an expression.
83.	D	If omitted, an implicit return happens at the end of the body.
84.	D	Write 5.0 / 9 to get floating-point division.
85.	A	The & goes in both the declaration and the definition.
86.	A	The function receives the memory location of the argument, so it must be a variable.
87.	A	Reference parameters change the arguments themselves.
88.	C	Forgetting & makes them call-by-value: inadvertent local variables. x and y don't change.
89.	B	Only reference parameters (with &) affect the arguments.
90.	B	a is a reference to x (becomes 7); b is a copy of y, so y stays 5.
91.	C	Both a and b refer to the same variable n, so it is incremented twice.
92.	A	That is the rule in Display 5.6.
93.	D	The function should not be called unless the precondition holds.
94.	C	Write pre/postconditions before designing how the function works.
95.	A	Functions cannot be nested, but they can call each other.
96.	B	Drivers call the function with test data and display the results.
97.	D	Stubs usually just return test values and are simple enough to trust.
98.	C	This way any bug found belongs to the function under test.
99.	D	#include . assert is used to check pre- and postconditions.
100.	A	Don't change code without understanding what you're doing.

Prepared from the supplied C++ Midterm Practice PDF. Question wording, order, answer choices, and answer-key letters are preserved; Question 99 receives a direct inline clarification because the source explanation is visibly incomplete.